

GuardShield-Runtime: Artifact-Aware Runtime Feedback for Progress-Preserving Safe Reinforcement Learning in Dense Traffic

Qin Jia^{1*}

^{1*}Institute of European and American Cultural Philosophy, Tianjin
Foreign Studies University, Tianjin, China.

Corresponding author(s). E-mail(s): y240252@stu.tjfsu.edu.cn;

Abstract

Safe reinforcement learning in dense traffic can produce misleading safety evidence when a policy lowers measured cost by suppressing motion rather than by regulating risk while maintaining route progress. We treat this failure as a closed-loop feedback artifact and propose GuardShield-Runtime, a simulator-level framework that couples a learned driving policy with runtime risk monitoring, bounded action intervention, and progress-safety diagnostics. Before execution, the framework monitors time-to-collision, speed, route progress, and lane-related signals to classify local risk, then bounds risk-triggered acceleration commands while logging each intervention. In MetaDrive dense-traffic simulations at density 0.15, the Risk-only baseline achieves 0.0% cost, but this apparent safety is accompanied by a 100.0% stop ratio and only 0.9% route completion. In contrast, the runtime-intervention variants preserve 85.7–86.9% route completion with 18.3–20.8% cost. A clean same-checkpoint ablation further shows that combining TTC and speed feedback reduces cost from 100.0% to 20.8% while raising route completion from 21.8% to 85.3%, whereas TTC-only and speed-only interventions remain less effective. These results suggest that dense-traffic safe reinforcement learning should be evaluated as monitored feedback regulation, where collision cost, route progress, stopping behavior, and intervention burden are interpreted jointly.

Keywords: safe reinforcement learning, runtime feedback intervention, artifact-aware evaluation, autonomous driving simulation, cybernetic control

1 Introduction

Dense autonomous-driving control is inherently a closed-loop system–environment interaction problem. A policy issues an action, surrounding traffic responds, the local state evolves, and the next action is selected under a changed risk condition. This feedback structure is central to cybernetics and control theory (Wiener 1948; Åström and Murray 2008), and it also creates the setting in which reinforcement-learning policies can produce misleading safety evidence. In dense simulated traffic, low measured cost can arise either because the ego vehicle regulates risk while making progress or because it stops and avoids interaction altogether. From a cybernetic perspective, the artifact arises because the learned controller can alter the system–environment interaction mode itself: from risk-regulated driving to non-interactive stopping.

This distinction matters because measured cost is often treated as evidence of safety in safe reinforcement learning. Standard deep reinforcement learning optimizes long-horizon return, while safe-RL methods augment this objective with cost constraints, penalties, or risk-sensitive terms (Altman 2021; García and Fernández 2015; Achiam et al. 2017; Chow et al. 2018). However, when safety is evaluated primarily through cost, a policy can exploit the objective by suppressing motion. In driving, this behavior should not be interpreted as safe task completion; it is task avoidance. Route progress, stop ratio, low-progress rate, time-to-collision (TTC) danger, and intervention frequency must therefore be measured alongside collision-oriented cost.

Runtime feedback intervention provides a complementary mechanism for addressing this failure. We do not treat a TTC rule or a speed cap as a final safety controller. Instead, we use them as auditable runtime feedback actuators for testing whether a safe-RL objective produces progress-preserving safety or merely lets the vehicle stop to avoid measured risk. This creates a monitored closed loop in which the policy proposes actions, the monitor estimates local risk, the intervention operator bounds risk-triggered acceleration commands, and the environment returns the next observation. The resulting loop can then be audited through progress-safety diagnostics.

To operationalize this idea, we introduce GuardShield-Runtime. The framework includes runtime-intervention variants that separate execution-side action bounding from reward-side risk shaping, allowing the diagnostic protocol to distinguish stationary reward exploitation from progress-preserving feedback regulation. We compare against artifact controls, including Risk-only and No-action guard, and external baselines, including PPO-Lagrangian and a TTC/RSS-style runtime filter. The claim is intentionally limited: GuardShield-Runtime is a simulator-level runtime feedback and diagnostic framework, not a proof-producing safety shield or a vehicle-grade real-time controller.

This paper makes four contributions. First, it formulates and empirically exposes the stationary safety artifact: cost-oriented safe-RL evaluation can be misleading when a policy suppresses motion instead of regulating risk while completing the route. Second, it proposes GuardShield-Runtime as a runtime feedback architecture that combines a PPO driving policy, TTC/speed/progress/lane monitoring, bounded longitudinal intervention, and auditable intervention logging. Third, it provides a clean mechanism test through same-checkpoint runtime ablation, showing that TTC-only and speed-only feedback are insufficient and that the combined operator supplies

the main execution-side improvement. Fourth, it packages artifact-aware evaluation around density stress, sensitivity, uncertainty, intervention burden, loop timing, and reproducibility artifacts so that collision cost, route progress, stopping behavior, TTC danger, and intervention frequency are interpreted together.

2 Related Work

2.1 Safe reinforcement learning and constrained policy optimization

Safe reinforcement learning seeks to optimize task reward while constraining or penalizing unsafe outcomes such as collisions, constraint violations, or unsafe exploration (García and Fernández 2015; Amodei et al. 2016; Leike et al. 2017). Constrained Markov decision processes, Constrained Policy Optimization, and related Lagrangian approaches formalize this idea by treating safety cost as a constraint or penalty during learning (Altman 2021; Achiam et al. 2017; Tessler et al. 2018). Recent infrastructure work further emphasizes the need for reproducible safe-RL benchmark implementations (Ji et al. 2024). Other approaches use Lyapunov arguments, model-based stability conditions, or safety critics to guide or restrict policy behavior (Berkenkamp et al. 2017; Chow et al. 2018; Dalal et al. 2018). These methods are important, but their simulator evidence can remain ambiguous in closed-loop driving when low cost is achieved through motion suppression rather than risk-regulated progress. The non-motion artifact can be viewed as a driving-specific instance of reward hacking or specification gaming, where optimizing a proxy objective yields behavior that satisfies the measured signal without satisfying the intended task (Amodei et al. 2016; Pan et al. 2022; Skalse et al. 2022). Our study therefore treats cost as one diagnostic signal among route completion, stop ratio, TTC danger, and intervention burden, rather than as the sole definition of safe driving.

2.2 Runtime shielding, safety filters, and vehicle rules

Runtime shielding modifies actions online when a monitored condition is violated (Alshiekh et al. 2018; Saunders et al. 2018). This idea is also related to runtime-assurance and Simplex-style architectures, where an advanced controller is supervised by a simpler fallback or safety controller (Seto et al. 1998). Recent runtime-assurance work similarly studies how to use learning to manage fallback-controller utilization while maintaining hard constraints (Miller et al. 2024). In driving, rule-based safety models such as RSS provide interpretable distance or TTC-style constraints (Shalev-Shwartz et al. 2017). Control barrier functions and reachability methods provide more formal languages for filtering or constraining actions (Ames et al. 2019; Fisac et al. 2019). GuardShield-Runtime belongs to this runtime-intervention family, but it addresses a different evidential question: whether action filtering reduces measured cost while preserving route progress, or merely hides a failure mode. Accordingly, we do not claim a formal invariant set or certified safe controller; instead, we study measurable runtime feedback intervention in a closed-loop simulator.

2.3 Cybernetic feedback and system–environment interaction

Cybernetics emphasizes closed-loop regulation through sensing, feedback, and action (Wiener 1948; Åström and Murray 2008). Reinforcement learning also operates through an action–environment–observation loop, but the optimized objective need not align with the desired closed-loop behavior. In our setting, the runtime monitor acts as a feedback sensor, the intervention operator acts as a bounded actuator, the learned policy plus runtime operator forms the controller, and progress-safety metrics serve as diagnostic outputs. This framing is useful because it makes the central failure observable and measurable: a policy may reduce cost by changing the closed-loop interaction from driving to stationary avoidance. GuardShield-Runtime uses this perspective to evaluate the learned policy not as an isolated function approximator, but as one component of a monitored feedback system.

2.4 Closed-loop autonomous-driving simulation

Simulation platforms support repeatable evaluation of policy learning, planning, and control, and autonomous-driving simulators such as CARLA and MetaDrive provide closed-loop traffic scenarios for testing learned driving policies (Dosovitskiy et al. 2017; Li et al. 2022). Deep RL has been explored for autonomous-driving control and decision making (Sallab et al. 2017; Kendall et al. 2019; Grigorescu et al. 2020; Kiran et al. 2021; Kuutti et al. 2021). However, closed-loop evaluation remains sensitive to scenario design, reward scale, baseline choice, random seed, and safety-case interpretation (Koopman and Wagner 2017; Henderson et al. 2018). Recent benchmark suites such as Safety-Gymnasium and SafeBench further show the need for standardized safety tasks, scenario coverage, and diagnostic metrics beyond a single cost scalar (Ji et al. 2023; Xu et al. 2022). Recent autonomous-driving evaluation work also emphasizes closed-loop assessment, scenario coverage, and safety-relevant evaluation rather than relying only on open-loop or aggregate metrics (Feng et al. 2023; Jia et al. 2024). MetaDrive is useful for this study because it provides compositional traffic scenarios together with route-completion diagnostics, making progress collapse visible under dense interaction (Li et al. 2022). We therefore use MetaDrive as a controlled diagnostic platform for testing whether runtime intervention reduces cost while preserving route progress under dense closed-loop interaction.

2.5 Positioning of this work

GuardShield-Runtime is positioned between training-time safe RL, runtime filtering, and diagnostic evaluation. Unlike reward-only risk shaping, it modifies the action before execution and logs each intervention. Unlike RSS or CBF-style formal filters, it is not presented as a proof of safety, but as a way to audit whether lower cost coincides with useful driving behavior. Its role is to test and document, through diagnostics, whether runtime feedback intervention can mitigate a non-motion artifact that cost-only safe-RL evaluation would otherwise hide. The contribution is therefore not a proof-producing safety method, but a diagnostic feedback architecture that makes proxy-objective failure observable under closed-loop driving evaluation. The closest prior mechanisms are TTC/RSS-style runtime filters, shielding and runtime-assurance

architectures, and reward-misspecification analyses. The present distinction is the combined focus on action-side feedback, motion-gated risk shaping, and artifact-aware progress diagnostics under the same simulator protocol.

3 GuardShield-Runtime Framework

3.1 Problem setting and closed-loop interface

Let o_t denote the simulator observation and let the learned policy propose a two-dimensional continuous action $a_t^\pi = (\delta_t, u_t^\pi)$, where δ_t denotes steering and u_t^π denotes the normalized acceleration/braking command. GuardShield-Runtime does not replace the learned policy; it inserts an intervention operator $\mathcal{I}_\theta$ between the policy output and the simulator execution interface:

$$a_t^{\text{exec}} = \mathcal{I}_\theta(o_t, a_t^\pi), \quad (1)$$

$$o_{t+1} \sim P(\cdot \mid o_t, a_t^{\text{exec}}). \quad (2)$$

The runtime layer uses observable wrapper-level signals available during closed-loop simulation: ego speed v_t , lane deviation d_t^{lane} , route-progress increment Δp_t , forward time-to-collision TTC_t , intervention events, and wall-clock loop timing. Table 1 defines how these signals enter the runtime monitor and how they are logged for diagnostics. This design keeps the intervention and diagnostics auditable at the wrapper level, rather than relying on post-hoc episode labels alone.

Table 1 Runtime signals used by the monitor and diagnostics.

Signal	Symbol	Role
Speed	v_t	speed guard and stop-ratio diagnostic
Route progress	Δp_t	low-progress diagnostic and Gated-risk motion gate
Lane deviation	d_t^{lane}	Gated-risk risk signal and lane diagnostic
Forward TTC	TTC_t	hard/soft risk state and TTC-danger diagnostic
Overspeed	ρ_t^{speed}	Gated-risk risk signal and speed-guard diagnostic
Intervention event	$\mathbf{1}[a_t^{\text{exec}} \neq a_t^\pi]$	intervention burden, reported as interventions per 100 steps

Core parameter values, action caps, speed-guard thresholds, and result-block provenance are reported in Appendix A.

3.2 Failure mode: stationary safety artifact

The key failure mode is what we call stationary safety: the policy appears safe under collision-oriented cost because it avoids motion. If a risk penalty or cost objective

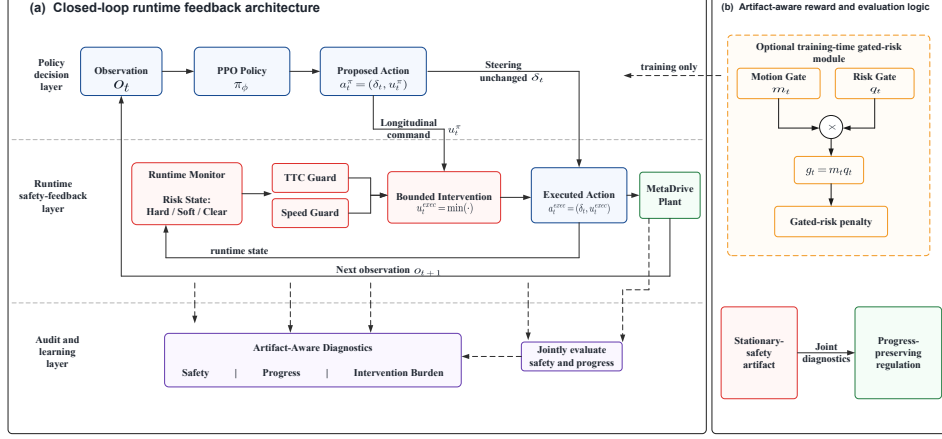


Fig. 1 Closed-loop GuardShield-Runtime architecture. The learned PPO backbone proposes an action, the runtime monitor observes TTC, speed, route, and lane signals before execution, the intervention operator bounds monitor-flagged acceleration commands, and the MetaDrive environment returns the next observation and diagnostic feedback.

primarily rewards the absence of collision without sufficiently requiring route progress, the learned policy can reduce measured cost by slowing or stopping. This is not merely a reporting inconvenience: it changes the interpretation of a safe-RL result. A method with low cost, near-zero route completion, and high stop ratio should not be interpreted as progress-preserving safe driving. For this reason, every main result reports cost together with success, route completion, low-progress rate, stop ratio, TTC-danger fraction, and intervention frequency.

3.3 Learning backbone and reward variants

The learned backbone is a PPO policy $\pi_\phi(o_t)$ (Schulman et al. 2017) trained on the native MetaDrive observation space with a two-dimensional continuous action for steering and longitudinal control. MetaDrive supplies the base task reward and termination signals; the risk-aware wrappers modify the step reward according to the variant configuration. When risk reward is enabled, the shaped objective combines scaled task reward with penalties for nonzero simulator cost, TTC risk, lane deviation, action variation, and overspeed. It also includes route-progress reward, idle penalty, success bonus, crash penalty, and out-of-road penalty according to the variant configuration, with detailed coefficients reported in Appendix A.

The Risk-only and No-action-guard artifact-control variants keep the risk-oriented reward but disable execution-side action modification. Guard and Shield use the same TTC-plus-speed runtime operator without continuous risk reward; they differ only in training context, with Guard trained under the curriculum schedule and Shield trained without curriculum. Gated-risk uses the same runtime operator and activates the continuous risk penalty only when the ego vehicle is in an active-control state and

a monitored risk signal is present:

$$m_t = \mathbf{1} \left[v_t \geq 3 \text{ km/h} \vee \Delta p_t > 10^{-4} \right], \quad (3)$$

$$q_t = \mathbf{1} \left[\rho_t^{\text{TTC}} \geq 0.01 \vee d_t^{\text{lane}} \geq 0.25 \vee \rho_t^{\text{speed}} > 0 \right], \quad (4)$$

$$g_t = m_t q_t. \quad (5)$$

Here, ρ_t^{TTC} denotes the wrapper-computed TTC-risk score and $\rho_t^{\text{speed}} = \max(v_t - v^*, 0) / \max(v^*, 10^{-6})$ denotes the overspeed signal relative to the target-speed band. Thus c_t^{risk} is used only through the gate g_t , while the simulator event cost remains logged separately as the binary episode-level safety outcome. The curriculum changes only the training density, map complexity, traffic/event probability schedule, and horizon schedule; it does not change the runtime intervention operator used in diagnostic evaluation.

3.4 Runtime monitor

The runtime monitor estimates a forward TTC signal from nearby vehicles that are ahead of the ego vehicle and closing in relative speed. Let $\mathcal{F}_t$ denote the monitored set of forward vehicles with positive longitudinal distance, acceptable lateral offset, and positive relative closing speed. For vehicle $i \in \mathcal{F}_t$, let $d_i(t)$ be the forward distance and $v_{\text{ego}}(t) - v_i(t)$ be the relative closing speed. The monitored TTC is

$$\text{TTC}_t = \begin{cases} \min_{i \in \mathcal{F}_t} \frac{d_i(t)}{v_{\text{ego}}(t) - v_i(t) + \epsilon}, & \mathcal{F}_t \neq \emptyset, \\ +\infty, & \mathcal{F}_t = \emptyset. \end{cases} \quad (6)$$

The small ϵ prevents numerical instability. If no closing forward vehicle is available, the monitor assigns the clear TTC state rather than creating a spurious collision risk. The implementation also reports the continuous TTC-risk score ρ_t^{TTC} , which increases as TTC_t falls below the stored TTC threshold. Hard, soft, and clear states are determined by τ_{hard} and τ_{soft} ; Section 5.5 evaluates sensitivity to the TTC threshold independently from the core diagnostic run. A dangerous step is counted when the monitored TTC is below the diagnostic threshold used by the logger.

3.5 Intervention operator

In the reported implementation, the intervention operator acts on the longitudinal command and leaves the policy steering command unchanged. It first applies a TTC-based acceleration cap:

$$u_t^{\text{guard}} = \begin{cases} \min(u_t^\pi, u_{\text{hard}}), & \text{TTC}_t < \tau_{\text{hard}}, \\ \min(u_t^\pi, u_{\text{soft}}), & \tau_{\text{hard}} \leq \text{TTC}_t < \tau_{\text{soft}}, \\ u_t^\pi, & \text{otherwise}, \end{cases} \quad (7)$$

where $u_{\text{hard}} = -1.0$, $u_{\text{soft}} = -0.6$, and τ_{base} is the stored TTC threshold. The hard and soft thresholds are $\tau_{\text{hard}} = \max(3.5, 0.60\tau_{\text{base}})$ and $\tau_{\text{soft}} = \max(6.0, \tau_{\text{base}})$. A speed guard then limits acceleration when the ego speed is above the target-speed band. With target speed v^* , the speed cap is

$$u^{\text{speed}}(v_t) = \begin{cases} -0.35, & v_t > v^* + 5, \\ 0, & v_t > v^*, \\ +\infty, & \text{otherwise,} \end{cases} \quad (8)$$

and the executed longitudinal command is

$$u_t^{\text{exec}} = \min\{u_t^{\text{guard}}, u^{\text{speed}}(v_t)\}, \quad a_t^{\text{exec}} = (\delta_t, u_t^{\text{exec}}). \quad (9)$$

The operator is per-step and memoryless in the reported implementation. It does not optimize a trajectory or certify future safety; it enforces a per-step command bound before the simulator step and logs whether $a_t^{\text{exec}} \neq a_t^\pi$. Intervention frequency is therefore reported as interventions per 100 control steps. A high value is not automatically better or worse: it indicates how often the runtime layer overrides the proposed command under the monitored conditions.

3.6 Diagnostic scope and relation to TTC/RSS filtering

GuardShield-Runtime is close to TTC/RSS-style runtime filtering because both modify commands before environment execution. The distinction is diagnostic and architectural rather than certification-based. The external RSS/TTC comparator is a single rule-based runtime filter driven by distance or TTC thresholds. GuardShield-Runtime instead treats the intervention as one component of a closed-loop evidence system: TTC danger, stop ratio, low-progress episodes, route completion, intervention frequency, and loop timing are reported together so that lower cost cannot be mistaken for useful driving when progress collapses.

The Gated-risk variant also addresses a reward-side failure mode that a pure execution filter does not target. Its risk penalty is active only when speed or route progress indicates an active control state, which is intended to reduce stationary exploitation of a risk objective. This scope means that the TTC cap should not be read as a certified safety rule. Instead, the framework uses runtime action intervention together with artifact-aware diagnostics to test whether dense-traffic safe-RL evidence reflects progress-preserving feedback regulation or conservative execution filtering alone.

3.7 Variant definitions

Table 2 summarizes the compared variants by separating the policy backbone, reward-side risk shaping, action-side intervention, speed guard, and motion/progress gate. **PPO backbone** is the unmodified learned policy and serves as the lower-bound runtime comparator. **Risk-only** uses risk reward without action intervention and is included to test whether low cost can be achieved through stationary avoidance. **No-action guard** keeps the risk-oriented reward but disables execution-side modification,

testing whether reward shaping alone can recover progress-preserving low-cost behavior. **Guard** and **Shield** apply the same TTC-plus-speed runtime operator without continuous risk reward, while **Gated-risk** combines runtime intervention with the motion/risk-gated reward shaping defined by g_t above.

This gate is designed to reduce stationary exploitation of a continuous risk penalty: if the agent stops, the risk penalty is no longer allowed to masquerade as progress-preserving behavior. Guard and Shield share the same execution-side operator but differ in training context; they should therefore be read as deployment variants rather than a clean single-factor ablation. The clean same-checkpoint runtime-operator contrast is provided separately in the Results section.

Table 2 Compared methods and runtime deployment variants.

Method	Training or reward	Runtime operator	Role in protocol
PPO backbone	No risk reward	None	Fixed policy backbone and lower-bound comparator
Risk-only	Risk reward	None	Tests whether low cost is stationary avoidance
No-action guard	Risk reward	Disabled	Reward-only control without execution-side action intervention
RSS/TTC filter	No risk reward	External TTC/RSS rule	Classical runtime-filter comparator
PPO-Lagrangian	Lagrangian penalty	None	Training-time constrained baseline under this protocol
Guard	Curriculum PPO, no risk reward	TTC hard/soft cap + speed guard	Runtime deployment variant with action intervention
Shield	No-curriculum PPO, no risk reward	Same TTC hard/soft cap + speed guard	Runtime deployment variant isolating the execution-side operator
Gated-risk	Curriculum PPO with gated risk	Same TTC hard/soft cap + speed guard	Adds motion/risk-gated reward to reduce stationary exploitation

Guard and Shield share the same execution-side operator but differ in training context; they are deployment variants rather than a clean single-factor ablation. The clean same-checkpoint runtime-operator contrast is reported separately in Table 4.

3.8 Algorithmic summary

Algorithm 1 GuardShield-Runtime closed-loop execution

Require: Learned policy π_ϕ , monitor thresholds θ , MetaDrive environment

- 1: **for** each control step t until termination **do**
 - 2: Observe o_t and wrapper signals $v_t, \Delta p_t$, lane state, and monitored nearby vehicles.
 - 3: Query the learned policy for $a_t^\pi = (\delta_t, u_t^\pi)$.
 - 4: Estimate TTC_t and classify the runtime state as hard, soft, or clear.
 - 5: Apply $\mathcal{I}_\theta(o_t, a_t^\pi)$ to obtain a_t^{exec} .
 - 6: Execute a_t^{exec} in the simulator.
 - 7: Log $a_t^\pi, a_t^{\text{exec}}$, risk state, cost, route, stop, low-progress, TTC-danger, intervention, and loop-time diagnostics.
 - 8: **end for**
 - 9: Aggregate diagnostics by episode, training seed, traffic density, and sensitivity setting.
-

4 Experimental Protocol

4.1 Environment and task

All experiments use MetaDrive as a controlled closed-loop dense-traffic driving simulator. The policy outputs continuous control commands and is evaluated using route progress, task success, collision or off-road cost, stop ratio, low-progress rate, speed, TTC-danger fraction, intervention frequency, and wall-clock simulation-loop timing. Success denotes the simulator episode-success flag for completing the driving task without terminal failure. Cost is reported as the percentage of episodes with a nonzero safety-cost event, which in this protocol is mainly caused by collision or out-of-road termination. Traffic density denotes the MetaDrive `traffic_density` setting that controls background-traffic generation. The main diagnostic density is 0.15, and the density-stress study additionally evaluates 0.08, 0.20, and 0.25. Diagnostic episodes terminate on simulator success, simulator failure, or the fixed 1500-step horizon.

4.2 Compared methods

The compared rows are defined in Table 2. PPO backbone is the unmodified learned policy and serves as the lower-bound comparator for runtime intervention. Risk-only and No-action guard are artifact-control variants that test whether risk-oriented reward shaping alone can reduce cost without producing useful route progress. Guard, Shield, and Gated-risk are the proposed runtime-intervention variants. The PPO-Lagrangian row is a tested training-time constrained baseline under the same environment family and seed protocol; its zero-success result is reported as a protocol outcome, not as a universal claim about all constrained RL implementations. The RSS/TTC filter is a meaningful external runtime comparator because it also modifies behavior through an interpretable distance or TTC rule before execution.

4.3 Training protocol and checkpoint selection

Training uses 16 parallel environments. The reported PPO-family checkpoints are trained for one million timesteps per seed with horizon 1500 and `ent_coef` = 0.01. Each density-0.15 aggregate reports three independently trained seeds. Unless otherwise stated, each seed contributes 40 diagnostic episodes, for 120 episodes per aggregate row. The default target speed for the proposed runtime variants is 18 km/h, and the stored reward/action-guard TTC threshold for the core configuration

is 12.0 s. The external RSS/TTC comparator is parameterized separately with soft and hard TTC thresholds of 6.0 s and 3.5 s and a 20 km/h target speed. Checkpoints are fixed before diagnostic evaluation, and no episode-level post-hoc filtering is used to select favorable outcomes. Full PPO, PPO-Lagrangian, RSS/TTC, and runtime-variant hyperparameters are listed in Appendix A, with run-level configuration files retained in the supplementary package.

4.4 Diagnostic evaluation protocol

Diagnostic evaluations are run as separate per-seed jobs so that episode-level TTC, speed, stop, intervention, route, and latency statistics remain traceable. Each density-0.15 aggregate row uses 120 diagnostic episodes across three training seeds, with 40 episodes per seed. Density stress applies the same diagnostic logic across four traffic densities. Sensitivity sweeps are independent diagnostic reruns; they should therefore be interpreted as trend evidence rather than exact numerical reproductions of the core default row. Appendix A reports the detailed parameter settings and result-block provenance.

4.5 Statistical reporting

Main tables report one decimal place for percent metrics. Appendix B reports Wilson intervals for binary outcomes and seed-level mean/range summaries for the density-0.15 comparison; the full bootstrap and per-seed CSVs remain in the supplementary package. These intervals document uncertainty within the simulator diagnostic protocol. Because the study uses three independently trained seeds, we do not use the intervals to claim universal policy-family dominance or real-world validity.

4.6 Supplementary artifacts

The supplementary package includes anonymized episode-level results, aggregate tables, plotting scripts, table-generation scripts, and configuration files required to reproduce the reported figures and tables. The package is organized by evidence block, matching the core diagnostics, external baselines, density stress, sensitivity sweeps, runtime diagnostics, and uncertainty summaries reported in the manuscript.

5 Results

5.1 Stationary safety artifact under cost-only evaluation

Low measured cost does not necessarily indicate progress-preserving safe behavior. Figure 2 and Table 3 show that Risk-only obtains 0.0% cost at density 0.15, but this apparent safety is accompanied by 0.9% route completion, 100.0% stop ratio, and 100.0% low-progress episodes. The PPO backbone shows the opposite failure mode: it continues to move, but it reaches 100.0% cost with only 20.7% route completion. The artifact is therefore a substantive evaluation failure rather than a reporting detail. It becomes visible only when motion and progress diagnostics are reported next to collision-oriented cost.

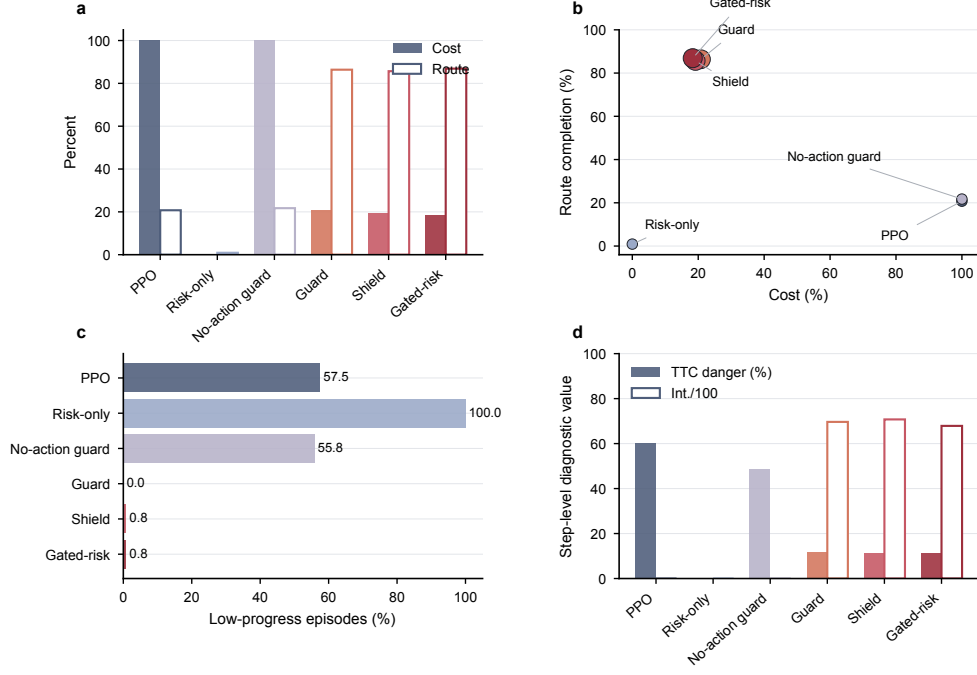


Fig. 2 Core artifact-aware evidence at density 0.15. (a) Cost and route completion separate stationary cost reduction from progress-preserving behavior. (b) The progress-safety frontier separates PPO backbone, Risk-only, reward-only control, and runtime intervention. (c) Low-progress rate exposes the non-motion artifact. (d) Two step-level diagnostics are reported on a common 0–100 scale: TTC-danger percentage and interventions per 100 control steps.

The PPO backbone is not selected as an artificially weakened baseline. It is the fixed learned policy used by the runtime-filter rows and is reported as a lower-bound/backbone comparator. Its zero-success result reflects the dense diagnostic success criterion: the policy still moves, with 20.7% route completion and 41.1 km/h mean speed at density 0.15, but frequent collision or out-of-road cost prevents successful completion. The same sanity check applies to PPO-Lagrangian, which reaches 22.1% route completion and 39.4 km/h mean speed but terminates through collision and out-road events. Risk-only is different: it has no collision or out-road cost because it almost does not move. This sanity check motivates the inclusion of the external RSS/TTC filter as a runtime-filter comparator, rather than relying only on PPO-family training baselines.

5.2 Same-checkpoint evidence for execution-side intervention

The improvement is not explained by reward shaping alone. No-action guard retains the risk-oriented objective but disables execution-side action intervention; it remains at 100.0% cost. Guard, Shield, and Gated-risk instead reduce cost to 18.3–20.8% while

Table 3 Artifact-aware evaluation at density 0.15. Cost-only evaluation incorrectly favors the stationary Risk-only baseline, whereas runtime-intervention variants preserve route progress while reducing cost.

Method	Success	Cost	Route	Low-prog.	Stop	TTC-danger	Int./100 steps
PPO backbone	0.0	100.0	20.7	57.5	0.1	60.4	0.0
Risk-only	0.0	0.0	0.9	100.0	100.0	0.0	0.0
No-action guard	0.8	100.0	21.7	55.8	0.2	48.7	0.0
Guard	59.2	20.8	86.4	0.0	2.4	11.8	69.6
Shield	62.5	19.2	85.7	0.8	2.8	11.4	70.8
Gated-risk	61.7	18.3	86.9	0.8	3.5	11.4	67.9

Values are percentages except Int./100 steps. Higher success and route are preferred; lower cost, low-progress, stop, and TTC-danger are preferred. Full seed scatter and loop timings are in Appendix Tables.

preserving 85.7–86.9% route completion. This comparison suggests that reward shaping alone is insufficient to explain the improvement. Under this comparison, the key difference is whether the proposed longitudinal command is bounded before execution.

Table 3 establishes the artifact and the deployment-level effect of the runtime-intervention family. Table 4 is the primary mechanism evidence because it fixes the policy checkpoints and changes only the runtime operator. This removes the main training-context confound. TTC-only and speed-only interventions both improve the backbone, but neither single operator matches the combined intervention. The TTC cap reduces TTC-danger from 58.2% to 10.5%, but it retains 65.0% cost and 56.3% route completion. The speed guard raises route completion to 66.2%, but it leaves TTC-danger at 32.7% and cost at 65.0%. Combining TTC and speed guards reduces cost to 20.8%, preserves 85.3% route completion, and reduces low-progress episodes to 1.7%. The mechanism is therefore not explained by a single speed limit or a single TTC cap; under the same-checkpoint protocol, TTC and speed feedback are complementary.

Table 4 Clean same-checkpoint runtime-intervention ablation at traffic density 0.15. All rows evaluate the same three frozen PPO-backbone checkpoints under the same 120-episode diagnostic protocol; only the runtime operator is changed.

Method	Succ.	Cost	Route	Coll.	Out-road	Stop	Low-prog.	TTC	Int./100
PPO backbone	0.0	100.0	21.8	59.2	40.8	0.2	51.7	58.2	0.0
PPO + TTC	29.2	65.0	56.3	28.3	36.7	2.9	25.8	10.5	11.4
PPO + Speed	34.2	65.0	66.2	61.7	3.3	0.7	9.2	32.7	82.8
PPO + Both	59.2	20.8	85.3	18.3	2.5	3.1	1.7	13.0	67.8

Because Guard, Shield, and Gated-risk are not strict single-policy ablations, we interpret them as a runtime-intervention family and rely on Table 4 for the clean operator contrast. The high intervention rate further shows that the learned backbone has not internalized the guard; execution-side feedback mitigates monitor-flagged proposals under the reported simulator protocol.

Table 5 External baseline comparison at density 0.15.

Method	Success	Cost	Route	Collision	Out-road	TTC-danger	Int./100 steps
PPO backbone	0.0	100.0	20.7	63.3	36.7	60.4	0.0
RSS/TTC filter	53.3	44.2	78.9	41.7	2.5	13.4	71.7
PPO-Lagrangian	0.0	100.0	22.1	69.2	30.8	60.1	0.0
Guard	59.2	20.8	86.4	18.3	2.5	11.8	69.6
Shield	62.5	19.2	85.7	15.8	3.3	11.4	70.8
Gated-risk	61.7	18.3	86.9	18.3	0.0	11.4	67.9

Values are percentages except Int./100 steps. RSS/TTC is an interpretable runtime-filter comparator; intervention frequency is diagnostic rather than an objective by itself.

5.3 Comparison with runtime and constrained-RL baselines

Table 5 shows that the TTC/RSS-style filter is a nontrivial runtime comparator: relative to the PPO backbone, it improves success to 53.3% and reduces cost to 44.2%. However, Guard, Shield, and Gated-risk reduce cost further, to 18.3–20.8%, while preserving 85.7–86.9% route completion. RSS/TTC also has a similar intervention frequency to the proposed variants, with 71.7 interventions per 100 steps versus 67.9–70.8, so the remaining gap is not simply caused by intervening more often. The RSS/TTC comparator uses its own interpretable threshold configuration, with soft/hard TTC thresholds of 6.0 s and 3.5 s and a target speed of 20 km/h, rather than an optimized sweep over the proposed framework’s threshold. It should therefore be read as an interpretable runtime-filter comparator, not as a tuned upper bound on rule-based filtering. The corresponding bar-chart visualization is retained in the supplementary package.

In this implementation and diagnostic setting, the PPO-Lagrangian row did not recover a progress-preserving low-cost behavior. Its cost limit, Lagrangian initialization, and update settings are reported in Appendix A; the evaluated policy was not stationary, with 22.1% route completion and 39.4 km/h mean speed, but it failed through collision and out-road events. The RSS/TTC and PPO-Lagrangian manifests are retained as a baseline audit so that these rows are not read as undocumented cherry-picked failures. We therefore treat the PPO-Lagrangian result as a protocol-specific constrained-baseline outcome rather than as a general statement about training-time constrained RL.

5.4 Density stress and operating envelope

Density stress exposes an operating envelope rather than a universal safety guarantee. At density 0.08, runtime variants achieve high success and low cost; at density 0.25, success drops and cost rises. Fig. 3 reports fixed-variant trends across traffic densities, while Appendix D reports the full density diagnostics and compact operating-envelope summary. Across the tested densities, runtime variants improve the progress-safety trade-off over PPO, but all variants degrade at density 0.25. This degradation is an important part of the evidence because it identifies the operating envelope of the reported runtime intervention.

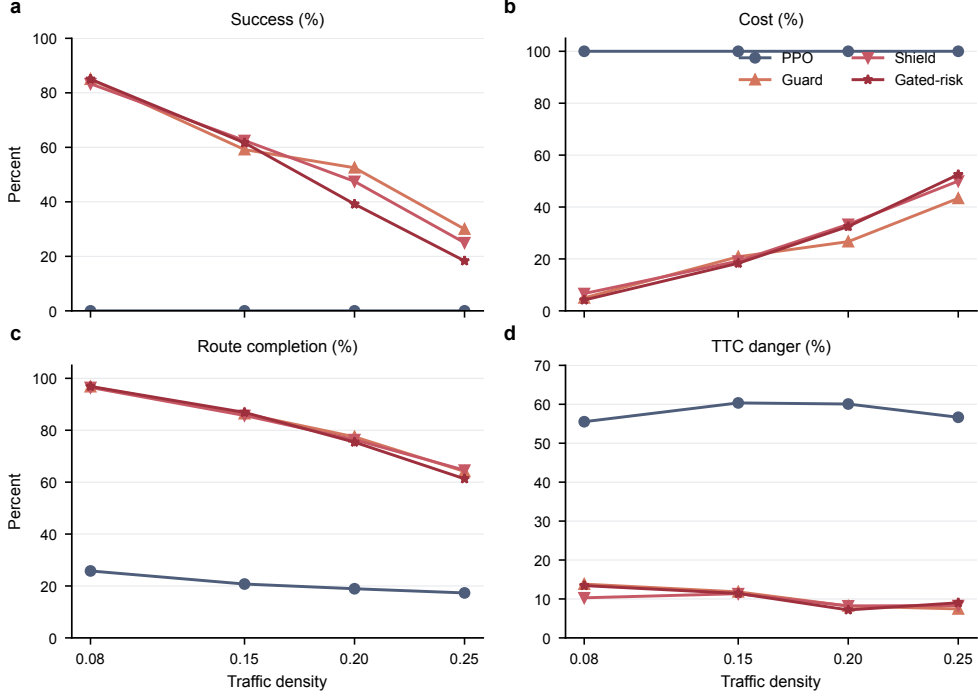


Fig. 3 Traffic-density stress test. Runtime variants preserve progress-safety better than the PPO backbone at the tested densities, but the highest density remains difficult.

5.5 Threshold sensitivity

The TTC-threshold sweep in Fig. 4 tests whether the main result depends on a single fixed threshold. Conservative thresholds can reduce cost, but they also change route completion and intervention frequency, which is expected for a runtime controller. The sensitivity rows are independent diagnostic reruns, not duplicates of the Table 3 default run; even a row with the same nominal threshold should therefore be read as trend evidence rather than as an exact numerical repeat. The sweep supports a trade-off interpretation rather than a single universally optimal threshold. The target-speed sweep is reported in Appendix F as a secondary sensitivity result.

5.6 Simulation-loop timing diagnostics

Figure 5 reports policy inference p95, full control-loop p95, and intervention frequency across traffic densities. These timing values are measured over realized simulator trajectories, so they are affected by episode length, density, termination mode, and simulator workload. They should therefore not be interpreted as a pure microbenchmark of the guard computation. The measurements are Python simulation-loop diagnostics, not vehicle-grade real-time certification. Full per-seed and per-density timing tables remain in supplementary CSV form.

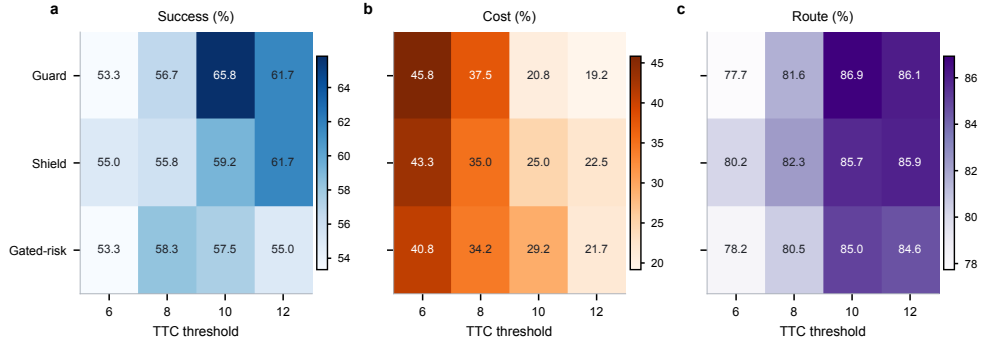


Fig. 4 TTC-threshold sensitivity at density 0.15. Heatmaps report success, cost, and route completion for Guard, Shield, and Gated-risk. The heatmap is an independent rerun; the TTC=12 cell is a same-nominal configuration but not the stored core-evaluation row in Table 3. The sweep shows threshold-dependent trade-offs rather than a single universally optimal setting.

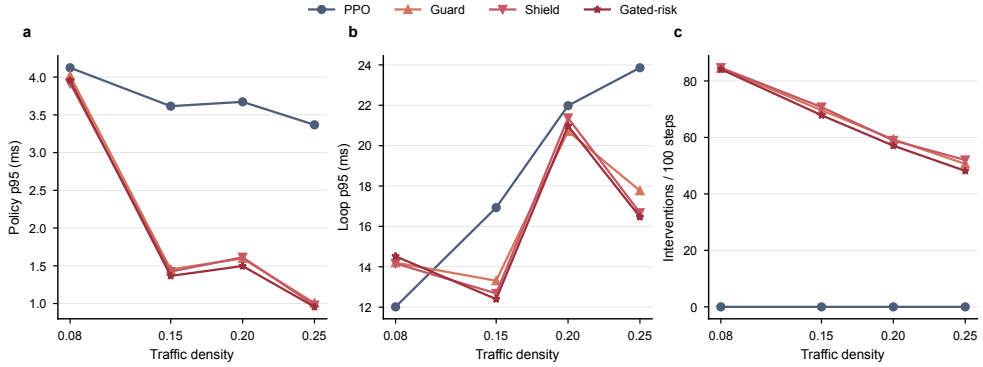


Fig. 5 Simulation-loop wall-clock diagnostics, not vehicle-grade real-time certification. The panels separate policy inference p95, full control-loop p95, and intervention frequency across traffic densities.

6 Discussion

The central finding is that cost-only safe-RL evaluation can be misleading in dense driving simulation. Risk-only appears safe under collision-oriented cost, but it is not a useful driving policy because it nearly stops. GuardShield-Runtime changes the closed-loop behavior by moving part of safety regulation from reward optimization to runtime feedback intervention. This makes the policy’s proposed command, the executed command, and the resulting progress-safety outcomes jointly observable, so that lower cost can be interpreted together with route progress, stopping behavior, and intervention burden.

6.1 Cybernetic interpretation

The results can be read as a feedback-regulation problem. The runtime monitor serves as the sensor, the intervention operator serves as the actuator, the simulator response supplies feedback, and progress-safety diagnostics are the closed-loop outputs. Under

this interpretation, the Risk-only failure is not simply a weak baseline. It is a degenerate feedback loop in which the policy reduces measured cost by avoiding interaction rather than regulating risk while making progress.

Table 6 Feedback-loop interpretation of GuardShield-Runtime.

Cybernetic role	GuardShield-Runtime component
Sensor	TTC, speed, route-progress, and lane-state monitor
Actuator	Bounded acceleration intervention
Plant/environment	MetaDrive dense-traffic simulator
Controller	Learned PPO policy plus runtime operator
Feedback output	Cost, route, stop, TTC-danger, intervention, and loop timing diagnostics

This cybernetic view also clarifies the role of the runtime variants. Guard and Shield alter the action before environment execution, so the feedback loop is changed at the actuator side. Gated-risk changes both the actuator side and the reward side, but only when motion or route progress indicates an active control state. The evidence therefore supports a bounded mechanism claim: simulator-level feedback intervention helps prevent a cost-only objective from being mistaken for progress-preserving low-cost behavior.

6.2 What the evidence supports

Table 7 summarizes the evidence boundary. The supported claims concern artifact diagnosis and simulator-level runtime intervention. The bounded claims concern density stress and simulation-loop timing. This boundary is methodologically important because the results are simulator diagnostics rather than formal safety guarantees. The method should be interpreted as a family of runtime feedback interventions, not as a single universally dominant controller. The fixed-variant density curves and sensitivity sweeps are therefore part of the main evidence rather than hidden in supplementary files.

Table 7 Claim-to-evidence summary.

Claim	Evidence	Location	Status
Cost-only safety is ambiguous	Risk-only obtains 0.0 cost but 100.0 stop and 0.9 route	Table 3, Fig. 2	Supported
Execution-side intervention matters	No-action guard remains at 100.0 cost; runtime variants reduce cost and preserve route	Table 3; Sec. 5.2	Supported
The tested TTC/RSS filter does not fully close the gap	The TTC/RSS-style filter improves PPO but retains 44.2 cost under the same diagnostic protocol	Table 5; supplementary external-baseline visualization	Supported
Benefit has an operating envelope	Cost rises and success drops at density 0.25	Table D1, Fig. 3	Bounded
Timing claim is simulation-loop only	Loop p95 is measured in the Python simulation loop	Fig. 5, App. F	Bounded

The table records supported and bounded claims under the reported simulator protocol.

6.3 Intervention burden and operating envelope

The runtime variants intervene in a large fraction of steps: approximately 68–71 interventions per 100 steps at density 0.15 and more than 80 interventions per 100 steps for the best low-density rows. This should not be interpreted as an ideal deployment property. It indicates that the learned backbone frequently proposes acceleration commands that the monitor flags as high risk under dense traffic. The present contribution is therefore runtime feedback mitigation and diagnostic exposure, not an autonomous policy that has internalized the shield.

The operating envelope is likewise bounded. Density stress shows that the runtime family remains useful at moderate density but degrades under the highest tested density. Sensitivity sweeps show that TTC threshold and target speed alter the balance among cost, route completion, and intervention frequency. Appendix E provides one qualitative rollout trace for failure-mode interpretation, with additional high-density traces retained in the supplementary package. These results argue for future training that reduces intervention burden by helping the learned policy internalize guard-compatible behavior, rather than treating high intervention frequency as a finished control solution.

6.4 Limitations and threats to validity

The main threats to validity are bounded but important. Internal validity is limited by the Guard/Shield training-context confound: they share the execution-side operator but are not a strict same-policy ablation. The same-checkpoint runtime ablation in Table 4 mitigates this concern for the execution operator, but it does not make the separately trained Guard, Shield, and Gated-risk policies into a strict single-policy ranking. Construct validity is limited by heuristic TTC thresholds and by simulator definitions of success, cost, low-progress, and TTC danger. External validity is limited because all evidence comes from MetaDrive simulation rather than real vehicles or hardware-in-the-loop control. Statistical validity is limited by three independently trained seeds; Appendix B reports episode-level intervals, while full seed ranges remain in the supplementary package. Runtime validity is limited because wall-clock measurements are Python simulation-loop timings rather than embedded real-time guarantees. The tested PPO-Lagrangian result is a baseline under this implementation and protocol, not a claim that all constrained RL methods fail. The step-level rollout visualization is a qualitative single-scenario diagnostic and should be read as a failure-mode illustration rather than aggregate evidence.

These limits also define the next research steps. Larger training-seed replication, richer traffic and map scenarios, a broader constrained-RL tuning sweep, and hardware-in-the-loop timing would strengthen the evidence boundary. High intervention frequency suggests a specific opportunity for future work: training the policy to internalize guard-compatible behavior and reduce reliance on execution-time correction.

7 Conclusion

This paper studied progress-preserving safe reinforcement learning in dense simulated traffic as a closed-loop feedback problem. The main finding is that cost-only evaluation can be misleading: a Risk-only policy can achieve zero measured cost by suppressing motion, rather than by regulating risk while completing the route. GuardShield-Runtime addresses this artifact by coupling a learned policy with runtime risk monitoring, bounded acceleration intervention, and progress-safety diagnostics. Across MetaDrive evaluations, runtime-intervention variants reduced cost while preserving route progress, and a same-checkpoint ablation showed that TTC and speed feedback are complementary rather than redundant. The evidence remains bounded to simulation and does not establish formal safety certification or real-world vehicle validation. The broader implication is that safe reinforcement learning for dense traffic should be evaluated as monitored feedback regulation, where cost, progress, intervention burden, and closed-loop behavior are interpreted jointly.

Supplementary information. The supplementary package includes anonymized episode-level results, aggregate tables, plotting scripts, table-generation scripts, and configuration files required to reproduce the reported figures and tables.

Declarations

Funding. The authors received no specific funding for this work.

Competing interests. The authors declare no competing interests.

Ethics approval and consent to participate. Not applicable. The study uses closed-loop driving simulation and does not involve human participants, human tissue, or animals.

Consent for publication. Not applicable.

Data availability. The anonymized submission package contains the raw episode CSVs, summary tables, generated publication figures, and source tables needed to reproduce the manuscript results. A public repository or archival DOI will be provided upon acceptance, subject to venue policy.

Code availability. The anonymized submission package contains the training, evaluation, diagnostic, table-generation, and figure-generation code used for the reported simulator study. A public repository or archival DOI will be provided upon acceptance, subject to venue policy.

Author contribution. Qin Jia is the sole author and is responsible for the study design, experiments, analysis, and manuscript preparation.

Appendix A Reproducibility and Evidence Map

This appendix organizes reproducibility material by manuscript claim. The goal is to make each reported figure or table traceable to its diagnostic block, parameter setting, and supplementary artifact. The printed tables provide the evidence map, default parameters, and result-block provenance; full episode-level files and table-ready aggregates are retained in the supplementary package.

Table A1 Evidence map and supplementary artifacts.

Evidence block	Manuscript support and main location	Supplementary artifacts
Core diagnostics	Non-motion artifact, core mechanism, and density-0.15 headline results in Fig. 2 and Table 3.	Episode CSVs and table-ready aggregates.
External baselines	Comparator evidence against runtime filtering and training-time penalty methods in Table 5.	Baseline diagnostic CSVs and the supplementary external-baseline figure.
Density stress	Operating-envelope evidence across densities 0.08, 0.15, 0.20, and 0.25 in Fig. 3, Table D1, and Appendix D.	All-density aggregate and episode CSVs.
Sensitivity	Independent diagnostic reruns for TTC threshold and target-speed trends in Fig. 4 and Appendix F.	Sensitivity aggregate and episode CSVs.
Runtime	Python simulation-loop timing and intervention-frequency evidence in Fig. 5 and Appendix F.	Runtime aggregate and per-seed CSVs.
Uncertainty and seed scatter	Episode-level intervals and seed variation at density 0.15 in Appendix B.	Wilson/bootstrap and seed-scatter CSVs.

The printed appendix groups the evidence package by claim rather than listing every generated file. Full raw and table-ready CSV files remain in the submission package.

Table A2 Compact default settings for training, intervention, and diagnostics.

Category	Key settings
Training protocol	1500-step horizon; 16 parallel SB3 environments; seeds 0/1/2; PPO MlpPolicy with Tanh [256, 256].
PPO optimizer	Learning rate 3×10^{-4} , $\gamma = 0.99$, GAE $\lambda = 0.95$, clip 0.2, 1024 rollout steps, batch 256, 10 epochs.
TTC intervention	Base TTC threshold 12.0 s; hard/soft thresholds 7.2/12.0 s; hard/soft acceleration caps $u_{\text{hard}} = -1.0$ and $u_{\text{soft}} = -0.6$.
Speed and motion gates	Target speed 18 km/h; speed guard caps acceleration at 0.0 above target speed and at -0.35 above target speed + 5 km/h; Gated-risk motion gate uses $v_{\text{min}} = 3.0$ km/h and $p_{\text{min}} = 10^{-4}$.
Diagnostic thresholds	Stop threshold $v_t < 1.0$ km/h; low-progress threshold route completion < 0.20 ; TTC-danger threshold 5.0 s; TTC closing-speed gate $\epsilon = 10^{-3}$.
External comparators	PPO-Lagrangian uses cost limit 0.25, $\lambda_0 = 20$, Lagrange lr 5, max 200, and window 20; RSS/TTC uses soft/hard TTC 6.0/3.5 s and target speed 20 km/h.

Detailed run provenance is reported in Table A4; sensitivity rows are independent diagnostic reruns and should not be numerically matched to the core rows.

Table A3 Baseline audit for external comparator rows.

Comparator	Evidence run and logged configuration	Interpretation boundary
PPO-Lagrangian	Three 1M-step training seeds with 16 parallel environments and the same diagnostic protocol. Logged settings include cost limit 0.25, $\lambda_0 = 20$, Lagrange lr 5, update window 20, retained manifests, and retained logs.	Protocol-specific constrained-RL baseline, not a universal claim that PPO-Lagrangian or CPO-style methods fail.
RSS/TTC filter	Three diagnostic seeds on frozen PPO checkpoints with an external action filter. Logged settings include soft/hard TTC thresholds of 6.0/3.5 s, 20 km/h target speed, and retained raw episode CSVs.	Interpretable runtime-filter comparator, not an optimized upper bound over all RSS/TTC thresholds.
TTC sensitivity	Independent Guard/Shield/Gated-risk diagnostic reruns across TTC thresholds. The TTC sweep is shown in Fig. 4; the target-speed sweep is retained in Appendix F.	Supports a trade-off interpretation and reduces dependence on a single selected threshold.

This audit records the baseline evidence available for this manuscript revision. It does not replace a future full constrained-RL or rule-filter hyperparameter search.

Table A4 Result-block provenance and parameter scope.

Block	Density	Parameter scope and source	Use in manuscript
Table 3 / Fig. 2	0.15	Core diagnostic episodes using 18 km/h target speed and TTC threshold 12.0 s.	Main artifact and mechanism result.
Fig. 4	0.15	Independent sensitivity rerun with stored speed settings and TTC thresholds 6/8/10/12.	Trend evidence only.
Fig. F1	0.15	Independent sensitivity rerun with target speeds 15/18/22 and TTC threshold 12.0 s.	Secondary trend evidence.
Table D1 / Fig. 3	0.08–0.25	Density-stress diagnostics using the stored variant configurations.	Operating-envelope evidence.

Rows with the same nominal TTC threshold can differ because sensitivity plots are independent diagnostic reruns, not duplicate evaluations of the core Table 3 rows.

Table A5 Software and hardware environment for the reported simulator runs.

Item	Value
Operating system	Ubuntu 22.04, Linux kernel 6.8.0-59-generic, x86_64
Python	3.11.15 in conda environment sb3
PyTorch	2.7.1+cu128
Stable-Baselines3	2.9.0a2, editable local checkout
Stable-Baselines3 commit	Source snapshot retained in the anonymized supplementary package; no public commit is disclosed during anonymous review
Project repository commit	Source snapshot retained in the anonymized supplementary package; no public commit is disclosed during anonymous review
MetaDrive	metadrive-simulator 0.4.3
Gymnasium	1.3.0
CPU	12th Gen Intel Core i5-12400F
GPU	NVIDIA GeForce RTX 4070 SUPER
Parallel environments	16
Training seeds	0, 1, 2

Appendix B Episode-Level Intervals and Seed Variation at Density 0.15

Appendix B reports episode-level uncertainty for the density-0.15 diagnostic comparison. These intervals make the comparison auditable, but they do not replace larger training-seed replication. Full seed-level ranges remain in the supplementary package; the wider Shield ranges support the family-level interpretation rather than a strict ranking among runtime variants.

Table B1 Episode-level uncertainty for binary outcomes at density 0.15.

(a) Task and artifact outcomes				
Method	Ep	Success	Cost	Low-progress
PPO backbone	120	0.0 (0.0–3.1)	100.0 (96.9–100.0)	57.5 (48.6–66.0)
Risk-only	120	0.0 (0.0–3.1)	0.0 (0.0–3.1)	100.0 (96.9–100.0)
No-action guard	120	0.8 (0.1–4.6)	100.0 (96.9–100.0)	55.8 (46.9–64.4)
RSS/TTC filter	120	53.3 (44.4–62.0)	44.2 (35.6–53.1)	2.5 (0.9–7.1)
PPO-Lagrangian	120	0.0 (0.0–3.1)	100.0 (96.9–100.0)	55.0 (46.1–63.6)
Guard	120	59.2 (50.2–67.5)	20.8 (14.5–28.9)	0.0 (0.0–3.1)
Shield	120	62.5 (53.6–70.6)	19.2 (13.1–27.1)	0.8 (0.1–4.6)
Gated-risk	120	61.7 (52.7–69.9)	18.3 (12.4–26.2)	0.8 (0.1–4.6)

(b) Event outcomes		
Method	Collision	Out-road
PPO backbone	63.3 (54.4–71.4)	36.7 (28.6–45.6)
Risk-only	0.0 (0.0–3.1)	0.0 (0.0–3.1)
No-action guard	53.3 (44.4–62.0)	46.7 (38.0–55.6)
RSS/TTC filter	41.7 (33.2–50.6)	2.5 (0.9–7.1)
PPO-Lagrangian	69.2 (60.4–76.7)	30.8 (23.3–39.6)
Guard	18.3 (12.4–26.2)	2.5 (0.9–7.1)
Shield	15.8 (10.4–23.4)	3.3 (1.3–8.3)
Gated-risk	18.3 (12.4–26.2)	0.0 (0.0–3.1)

Entries report point estimate with Wilson 95% interval in parentheses, in percentage points.

Appendix C Training Run Audit

The formal checkpoints come from completed $n_{\text{env}} = 16$ training runs whose configuration and result-block provenance are reported in Appendix A. Training curves are retained in the supplementary CSV and run logs for auditability, but they are not used for checkpoint selection or for the main mechanism claim. The quantitative conclusions in the paper are based on frozen-checkpoint diagnostic evaluation, same-checkpoint runtime ablation, seed-level summaries, and density stress rather than visual inspection of training curves.

Appendix D Full Density Operating-Envelope Diagnostics

The main text reports fixed-variant density trends in Fig. 3. Table D1 provides a compact lowest-cost observed runtime summary at each density, and Table D2 reports the full method-by-density diagnostics. Table D1 is included only as an operating-envelope summary; it should not be read as a claim that one fixed variant dominates across densities.

Table D1 Compact operating-envelope summary.

Dens.	PPO C	PPO R	Lowest-cost runtime row	C	R	Cost red.	Route gain
0.08	100.0	25.8	Gated-risk	4.2	96.9	95.8	71.1
0.15	100.0	20.7	Gated-risk	18.3	86.9	81.7	66.1
0.20	100.0	18.9	Guard	26.7	77.5	73.3	58.5
0.25	100.0	17.3	Guard	43.3	64.1	56.7	46.8

C and R denote cost and route completion in percent. This is a compact lowest-cost runtime summary at each density, not a claim that one fixed variant dominates across densities. Fig. 3 and Appendix D show the fixed-variant trends and operating-envelope table.

Table D2 Full density diagnostics across the tested operating envelope.

Dens.	Method	Succ.	Cost	Route	Coll.	Out	Low	Stop	TTC	Int.
0.08	PPO	0.0	100.0	25.8	43.3	56.7	42.5	0.1	55.5	0.0
0.08	Guard	85.0	5.0	96.8	2.5	2.5	0.0	0.5	13.8	84.5
0.08	Shield	83.3	6.7	96.5	6.7	0.0	0.0	0.5	10.3	84.7
0.08	Gated-risk	85.0	4.2	96.9	4.2	0.0	0.0	0.6	13.4	84.2
0.15	PPO	0.0	100.0	20.7	63.3	36.7	57.5	0.1	60.4	0.0
0.15	Guard	59.2	20.8	86.4	18.3	2.5	0.0	2.4	11.8	69.6
0.15	Shield	62.5	19.2	85.7	15.8	3.3	0.8	2.8	11.4	70.8
0.15	Gated-risk	61.7	18.3	86.9	18.3	0.0	0.8	3.5	11.4	67.9
0.20	PPO	0.0	100.0	18.9	68.3	31.7	67.5	0.2	60.1	0.0
0.20	Guard	52.5	26.7	77.5	26.7	0.0	2.5	5.3	8.2	59.3
0.20	Shield	47.5	33.3	76.4	30.8	2.5	0.0	5.1	8.2	58.9
0.20	Gated-risk	39.2	32.5	75.4	32.5	0.0	2.5	5.2	7.2	57.1
0.25	PPO	0.0	100.0	17.3	69.2	30.8	74.2	0.2	56.7	0.0
0.25	Guard	30.0	43.3	64.1	40.0	3.3	4.2	8.0	7.4	50.6
0.25	Shield	25.0	50.0	64.6	45.8	4.2	5.0	8.2	8.3	52.0
0.25	Gated-risk	18.3	52.5	61.3	51.7	0.8	4.2	8.4	9.0	48.2

Values are percentages except Int., which denotes interventions per 100 steps. PPO denotes the PPO backbone. Dens., Succ., Coll., Low, and TTC denote density, success, collision, low-progress, and TTC-danger. Artifact-control density rows remain in the supplementary CSV package.

Appendix E Qualitative Failure-Case Rollouts

Figure E1 reports a deterministic single-scenario rollout used only for failure-mode interpretation. The source files include per-step position, speed, lane deviation, TTC-risk, risk-gate, and intervention traces. Additional high-density qualitative rollout traces are provided in the supplementary package. These figures are not used for aggregate claims; the main density, ablation, and confidence-interval tables remain the quantitative evidence.

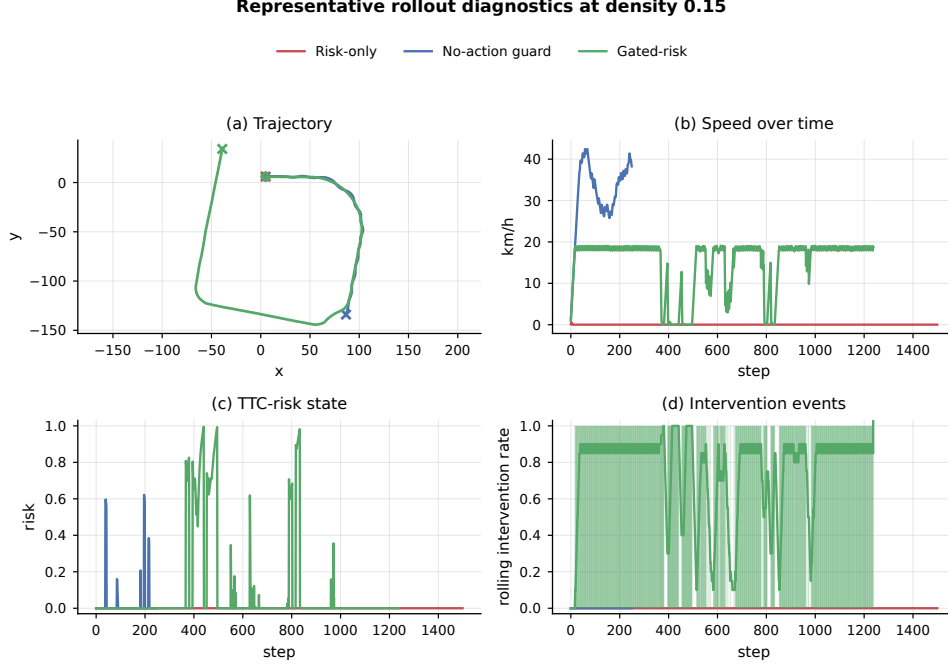


Fig. E1 Qualitative behavior rollouts at traffic density 0.15. The deterministic scenario illustrates three distinct closed-loop behaviors: Risk-only suppresses motion, No-action guard moves without execution-side intervention and terminates early, and Gated-risk maintains progress under frequent runtime intervention. The figure is used only for failure-mode interpretation.

Appendix F Sensitivity and Runtime Diagnostics

This appendix reports secondary diagnostics used to interpret parameter sensitivity and simulation-loop timing. These results support the operating-envelope discussion, but they are not the primary evidence for the main mechanism claim.

The target-speed sweep is included to check whether the runtime controller simply benefits from an arbitrary low-speed setting. The observed pattern shows a real progress-safety coupling: lower speed can reduce cost, while moderate speed often preserves success and route completion.

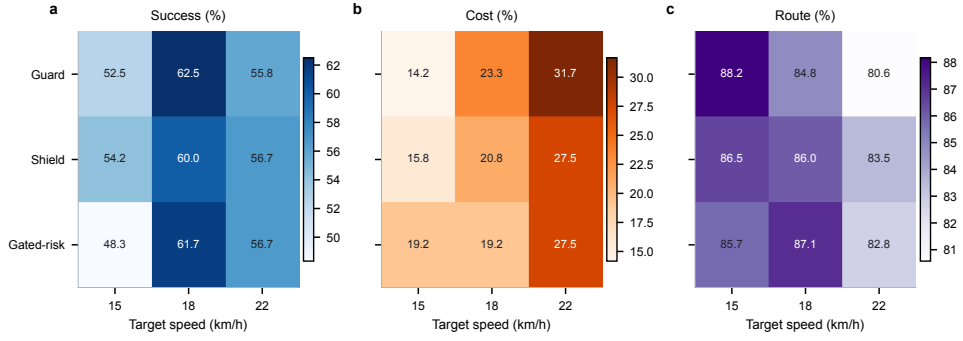


Fig. F1 Target-speed sensitivity at density 0.15. The sweep is reported as secondary sensitivity evidence because the main text focuses on TTC-threshold behavior.

Table F1 Compact sensitivity summary at density 0.15.

Variant	Best success	Lowest cost	Highest route	Interpretation
Guard	TTC=10: 65.8	speed=15: 14.2	speed=15: 88.2	Success, cost, and route optima are not identical.
Shield	TTC=12: 61.7	speed=15: 15.8	speed=15: 86.5	Success, cost, and route optima are not identical.
Gated-risk	speed=18: 61.7	speed=18: 19.2	speed=18: 87.1	Success, cost, and route optima are not identical.

The full TTC-threshold and target-speed sweeps remain in the supplementary CSV package. This table reports only the settings that optimize the three headline outcomes within the sweep.

References

- Alshiekh, M., Bloem, R., Ehlers, R., Könighofer, B., Niekum, S., Topcu, U.: Safe reinforcement learning via shielding. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, pp. 2669–2678 (2018). <https://doi.org/10.1609/aaai.v32i1.11797>
- Ames, A.D., Coogan, S., Egerstedt, M., Notomista, G., Sreenath, K., Tabuada, P.: Control barrier functions: Theory and applications. In: 2019 18th European Control Conference, pp. 3420–3431 (2019). <https://doi.org/10.23919/ECC.2019.8796030>
- Achiam, J., Held, D., Tamar, A., Abbeel, P.: Constrained policy optimization. In: Proceedings of the 34th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 70, pp. 22–31. PMLR, (2017). <https://proceedings.mlr.press/v70/achiam17a.html>

- Altman, E.: Constrained Markov Decision Processes. CRC Press, (2021). <https://doi.org/10.1201/9781315140223>
- Åström, K.J., Murray, R.M.: Feedback Systems: An Introduction for Scientists and Engineers. Princeton University Press, (2008). <https://fbsbook.org/>
- Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., Mané, D.: Concrete problems in ai safety. arXiv preprint arXiv:1606.06565 (2016) [arXiv:1606.06565](https://arxiv.org/abs/1606.06565)
- Berkenkamp, F., Turchetta, M., Schoellig, A.P., Krause, A.: Safe model-based reinforcement learning with stability guarantees. In: Advances in Neural Information Processing Systems, vol. 30 (2017). <https://papers.nips.cc/paper/6692-safe-model-based-reinforcement-learning-with-stability-guarantees>
- Chow, Y., Nachum, O., Duenez-Guzman, E., Ghavamzadeh, M.: A lyapunov-based approach to safe reinforcement learning. In: Advances in Neural Information Processing Systems, vol. 31 (2018). <https://papers.nips.cc/paper/8032-a-lyapunov-based-approach-to-safe-reinforcement-learning>
- Dalal, G., Dvijotham, K., Vecerik, M., Hester, T., Paduraru, C., Tassa, Y.: Safe exploration in continuous action spaces. arXiv preprint arXiv:1801.08757 (2018) [arXiv:1801.08757](https://arxiv.org/abs/1801.08757)
- Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: Carla: An open urban driving simulator. In: Proceedings of the 1st Annual Conference on Robot Learning. Proceedings of Machine Learning Research, vol. 78, pp. 1–16. PMLR, (2017). <https://proceedings.mlr.press/v78/dosovitskiy17a.html>
- Fisac, J.F., Akametalu, A.K., Zeilinger, M.N., Kaynama, S., Gillula, J., Tomlin, C.J.: Bridging hamilton-jacobi safety analysis and reinforcement learning. In: 2019 International Conference on Robotics and Automation, pp. 8550–8556 (2019). <https://doi.org/10.1109/ICRA.2019.8794107>
- Feng, S., Sun, H., Yan, X., Zhu, H., Zou, Z., Shen, S., Liu, H.X.: Dense reinforcement learning for safety validation of autonomous vehicles. Nature **615**(7953), 620–627 (2023) <https://doi.org/10.1038/s41586-023-05732-2>
- García, J., Fernández, F.: A comprehensive survey on safe reinforcement learning. Journal of Machine Learning Research **16**(1), 1437–1480 (2015)
- Grigorescu, S., Trasnea, B., Cocias, T., Macesanu, G.: A survey of deep learning techniques for autonomous driving. Journal of Field Robotics **37**(3), 362–386 (2020) <https://doi.org/10.1002/rob.21918>
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., Meger, D.: Deep reinforcement learning that matters. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32 (2018)

- Jia, X., Yang, Z., Li, Q., Zhang, Z., Yan, J.: Bench2Drive: Towards multi-ability benchmarking of closed-loop end-to-end autonomous driving. In: *Advances in Neural Information Processing Systems*, vol. 37 (2024). <https://arxiv.org/abs/2406.03877>
- Ji, J., Zhang, B., Zhou, J., Pan, X., Huang, W., Sun, R., Geng, Y., Zhong, Y., Dai, J., Yang, Y.: Safety-gymnasium: A unified safe reinforcement learning benchmark. In: *Advances in Neural Information Processing Systems*, vol. 36 (2023)
- Ji, J., Zhou, J., Zhang, B., Dai, J., Pan, X., Sun, R., Huang, W., Geng, Y., Liu, M., Yang, Y.: OmniSafe: An infrastructure for accelerating safe reinforcement learning research. *Journal of Machine Learning Research* **25**(285), 1–6 (2024)
- Kuutti, S., Bowden, R., Jin, Y., Barber, P., Fallah, S.: A survey of deep learning applications to autonomous vehicle control. *IEEE Transactions on Intelligent Transportation Systems* **22**(2), 712–733 (2021) <https://doi.org/10.1109/TITS.2019.2962338>
- Kendall, A., Hawke, J., Janz, D., Mazur, P., Reda, D., Allen, J.-M., Lam, V.-D., Bewley, A., Shah, A.: Learning to drive in a day. In: *2019 International Conference on Robotics and Automation*, pp. 8248–8254 (2019). <https://doi.org/10.1109/ICRA.2019.8793742>
- Kiran, B.R., Sobh, I., Talpaert, V., Mannion, P., Al Sallab, A.A., Yogamani, S., Pérez, P.: Deep reinforcement learning for autonomous driving: A survey. *IEEE Transactions on Intelligent Transportation Systems* **23**(6), 4909–4926 (2021) <https://doi.org/10.1109/TITS.2021.3054625>
- Koopman, P., Wagner, M.: Autonomous vehicle safety: An interdisciplinary challenge. *IEEE Intelligent Transportation Systems Magazine* **9**(1), 90–96 (2017) <https://doi.org/10.1109/MITS.2016.2583491>
- Leike, J., Martic, M., Krakovna, V., Ortega, P.A., Everitt, T., Lefrancq, A., Orseau, L., Legg, S.: Ai safety gridworlds. *arXiv preprint arXiv:1711.09883* (2017) [arXiv:1711.09883](https://arxiv.org/abs/1711.09883)
- Li, Q., Peng, Z., Feng, L., Zhang, Q., Xue, Z., Zhou, B.: Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **45**, 3461–3475 (2022) <https://doi.org/10.1109/TPAMI.2022.3190471>
- Miller, K., Zeitler, C.K., Shen, W., Hobbs, K., Schierman, J., Viswanathan, M., Mitra, S.: Optimal runtime assurance via reinforcement learning. In: *2024 ACM/IEEE 15th International Conference on Cyber-Physical Systems (ICCPS)*, pp. 67–76 (2024). <https://doi.org/10.1109/ICCPS61052.2024.00013>
- Pan, A., Bhatia, K., Steinhardt, J.: The effects of reward misspecification: Mapping and mitigating misaligned models. In: *International Conference on Learning*

- Representations (2022). <https://openreview.net/forum?id=JYtwGwIL7ye>
- Sallab, A.E., Abdou, M., Perot, E., Yogamani, S.: Deep reinforcement learning framework for autonomous driving. *Electronic Imaging* **2017**(19), 70–76 (2017) <https://doi.org/10.2352/ISSN.2470-1173.2017.19.AVM-023>
- Skalse, J., Howe, N.H.R., Krashennnikov, D., Krueger, D.: Defining and characterizing reward hacking. In: *Advances in Neural Information Processing Systems*, vol. 35, pp. 9460–9471 (2022)
- Seto, D., Krogh, B., Sha, L., Chutinan, A.: The simplex architecture for safe online control system upgrades. In: *Proceedings of the 1998 American Control Conference*, vol. 6, pp. 3504–3508 (1998). <https://doi.org/10.1109/ACC.1998.703255>
- Saunders, W., Sastry, G., Stuhlmüller, A., Evans, O.: Trial without error: Towards safe reinforcement learning via human intervention. In: *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, pp. 2067–2069 (2018)
- Shalev-Shwartz, S., Shammah, S., Shashua, A.: On a formal model of safe and scalable self-driving cars. *arXiv preprint arXiv:1708.06374* (2017) [arXiv:1708.06374](https://arxiv.org/abs/1708.06374)
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017) [arXiv:1707.06347](https://arxiv.org/abs/1707.06347)
- Tessler, C., Mankowitz, D.J., Mannor, S.: Reward constrained policy optimization. *arXiv preprint arXiv:1805.11074* (2018) [arXiv:1805.11074](https://arxiv.org/abs/1805.11074)
- Wiener, N.: *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press, (1948)
- Xu, C., Ding, W., Lyu, W., Liu, Z., Wang, S., He, Y., Hu, H., Zhao, D., Li, B.: Safebench: A benchmarking platform for safety evaluation of autonomous vehicles. In: *Advances in Neural Information Processing Systems*, vol. 35, pp. 25667–25682 (2022)